

Customer Churn Prediction System

End-to-End ML System with CRISP-DM & MLOps (MLflow)

Hector Alejandro Vargas Gutierrez

June 22, 2026

System Architecture

Platform Components

An end-to-end MLOps ecosystem designed for dynamic feature extraction, hyperparameter shifting, and localized storage.

- ▶ **Data Ingestion Pipeline:** Handles modular ingestion via structured CSV files securely bounded by project root configurations.
- ▶ **Feature Engineering Pipeline:** `DynamicFeatureEngineer` transforms raw inputs into interaction terms and indicators.
- ▶ **Adaptive ML Optimization:** Runs sequential multi-stage Optuna tuning loops with real-time MLflow Tracking metrics.
- ▶ **Model Registry:** Tracks parameters and logs serialized cloudpickle models to a dedicated SQLite storage layer.
- ▶ **API Inference Service:** Implements microservices using FastAPI to deliver endpoints for execution and runtime statistics.

Churn Prediction Workflow

Data Lifecycle Execution

From raw transaction parameters to calculated strategic customer retention campaigns.

1. **Customer Input Data:** Raw structured vectors pass through validation checks upon reaching API micro-routers.
2. **Feature Transformation:** Preprocessors construct feature names out, scaling variables using target encodings.
3. **Pipeline Type Defense:** An implicit `XGBNumericCaster` layer forces features into `float32` shapes to shield the base boosting trees.
4. **Model Inference:** High-velocity tree tracking models run predictive passes to assign probability weights.
5. **X-Ray Explanations:** Triggers localized tree-path dependent SHAP routines to serialize feature importances back to MLflow.

Adaptive Multi-Stage Optimization Loop

- Implements sequential search boundary adjustment loops over 4 key phases to scale parameter ranges dynamically:

Optimization Phase	Trials Run	Search Boundary Strategy
Base Phase	200	Explores baseline hyperparam defaults
Phase 2 Step	100	Quantile profiling and range movement
Phase 3 Step	50	Concentrates boundaries on optimum zones
Phase 4 Step	25	Fine-tunes localized high-perf vectors

Table: Sequential Parameter Convergence Lifecycle

Model Promotion & Champion Guardrails

The tracking server queries all parent runs inside the SQLite environment. The platform automatically promotes new models only if the calculated test PR-AUC surpasses the highest historical metric.

Microservice Interface Endpoints

The delivery layer exposes clean REST schemas managed by Uvicorn.

```
# Ingestion and Retraining
@app.post("/train")
async def train(file: UploadFile, experiment_name: str):
    # Runs ingestion, engineering, and multi-stage loops

# Real-Time Operational Predictor
@app.post("/predict")
async def predict(file: UploadFile, run_id: str):
    # Transforms arrays and produces XGBoost probabilities

# Analytics and Audit
@app.get("/experiments/best_runs")
def get_best_runs():
    # Returns history tracking logs via BestRunResponse
    Pydantic schema
```

Containerized Deployment Architecture

Reproducible Ecosystem Environment

The application is packaged inside a lightweight `python:3.12-slim` container, ensuring quick start-up times and isolation.

- ▶ **FastAPI Application Container (hop_api):** Exposes internal operational processing functions via port 8000.
- ▶ **MLflow Dashboard Service (hop_mlflow):** Mounts unified workspace directory binds to expose the visualization engine UI on port 5000.
- ▶ **Persistent Storage Layer:** Mounts localized shared system volumes to maintain tracking information inside `mlflow.db` and model plots under `mlartifacts/`.

Production Monitoring

Automated Drift Detection Framework

Production monitoring services detect data and target distribution shifts before model performance degradation becomes significant.

- ▶ **Feature Drift Detection:** KS tests for numerical variables and Chi-Square tests for categorical variables.
- ▶ **Pipeline-Aware Monitoring:** Drift evaluated after feature engineering and preprocessing transformations.
- ▶ **Target Drift Analysis:** Monitoring of churn-rate shifts and class distribution changes.
- ▶ **Monitoring APIs:** Real-time drift assessment endpoints returning statistical reports and drift alerts.

Monitoring Service Endpoints

The observability layer exposes statistical drift monitoring services for production model governance.

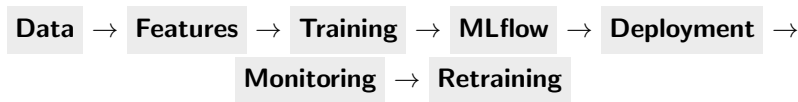
```
# Feature Distribution Monitoring
@app.post("/monitor/feature-drift")
async def monitor_drift(reference_file: UploadFile,
                        current_file: UploadFile, run_id: str):
    # Executes KS and Chi-Square drift tests
    # across transformed production features

# Target Distribution Monitoring
@app.post("/monitor/target-drift")
async def monitor_target_drift(reference_file: UploadFile,
                               current_file: UploadFile, target_column: str = "Exited"):
    # Evaluates churn-rate shifts and
    # class distribution drift statistics
```

End-to-End MLOps Pipeline

Continuous Integration & Training Loop

Unified architectural lifecycle mapping feedback dependencies from collection to iteration.



Challenges

- ▶ Class imbalance in churn data
- ▶ Feature leakage risks
- ▶ Model interpretability vs performance tradeoff

Future Improvements

- ▶ Real-time churn scoring
- ▶ Automated retraining pipeline
- ▶ A/B testing retention strategies
- ▶ Improved feature store integration

Conclusion

- ▶ Built end-to-end churn prediction system
- ▶ Achieved strong predictive performance ($F1 = XX\%$)
- ▶ Implemented full MLOps lifecycle with MLflow
- ▶ System ready for production decision support

Thank You